Nexus AI — Enterprise Knowledge Assistant

Project Report | Local RAG-based document question answering

Project type: Academic / portfolio prototype
Application: Streamlit web interface
Generation: Groq LLM API
Retrieval: Embeddings with FAISS similarity search

Abstract

Nexus AI is a document-grounded question-answering assistant designed to help users query policy PDFs. The application processes uploaded documents, extracts their text with page metadata, divides text into chunks, creates embeddings, and stores vectors in a FAISS index. At query time, it retrieves relevant chunks subject to document access metadata and supplies the allowed context to a Groq-hosted language model. Responses include page references and allow users to inspect retrieved sources. The prototype was developed and tested locally.

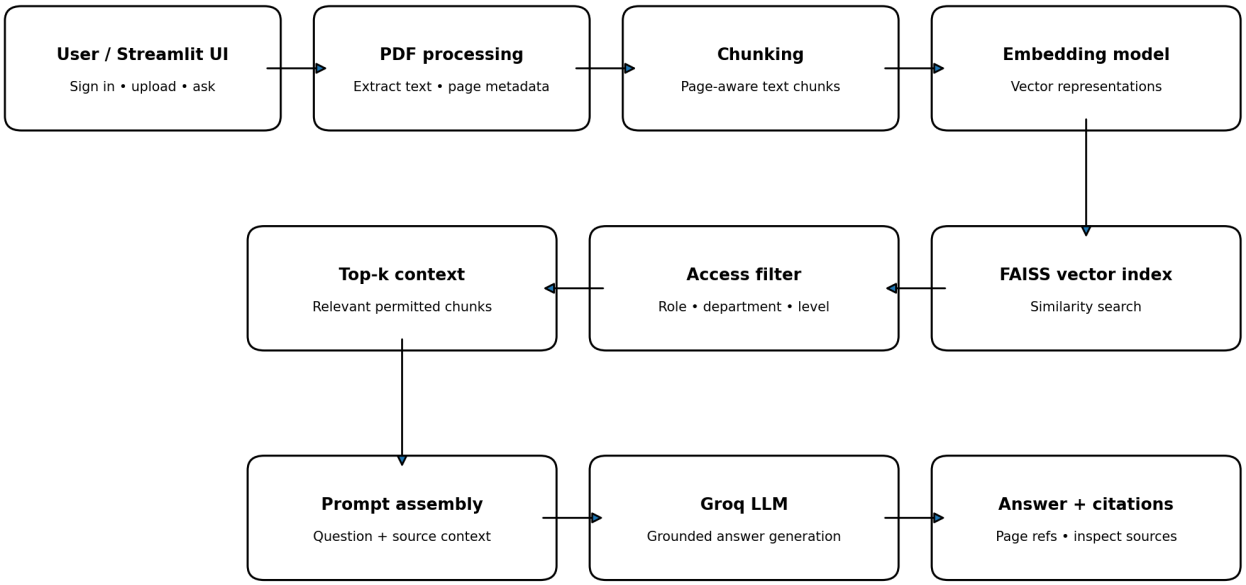
1. Problem Statement

Organizations store policies and operational information in lengthy documents. Finding exact answers manually can be time-consuming. A conventional language model may also produce unsupported answers if it is not grounded in source material. Nexus AI addresses this by retrieving relevant passages from user-provided documents and generating answers based on those passages, with source visibility.

2. Objectives

• Build a local document Q&A; interface for PDF policy files.
• Process multiple PDFs into searchable chunks.
• Retrieve semantically relevant passages using embeddings and FAISS.
• Generate context-grounded answers through Groq.
• Display page citations and retrieved source snippets.
• Enforce role, department, and access-level filters during retrieval.

3. System Architecture



Nexus AI — Local Enterprise RAG Knowledge Assistant

The main flow is: user upload → PDF extraction → chunking → embedding → FAISS indexing. During question answering, the query is embedded, relevant chunks are searched, access rules are applied, and permitted context is assembled for the LLM. The generated response is displayed with page citations.

4. Main Components

Streamlit UI: Handles user interaction, document upload, configuration, and chat display.
PDF loader: Extracts document text and page numbers.
Chunker: Splits extracted text into manageable segments with overlap.
Embedding model: Converts chunks and questions into vector representations.
FAISS: Supports vector similarity retrieval.
Access filter: Uses role, department, and access-level metadata to decide which chunks may be returned.
Groq LLM: Generates an answer from the supplied context.
Citations: Connect answers to source pages and retrieved chunks.

5. Access Control

Each uploaded document is associated with metadata such as department and access level. Retrieval checks this metadata before adding chunks to the answer context. In the current prototype, the Admin role has broad access across departments. Non-admin access depends on the configured department and access-level rules. During local testing, an Admin-only synthetic test document was accessible to the Admin account and not returned to the Employee account. A Finance-tagged synthetic document was retrieved for the Finance Manager account.

6. Testing and Results

Functional testing covered PDF processing, policy Q&A;, source citations, absent-information behavior, and access filtering. Direct questions about remote-work limits, core hours, collaboration days, and travel rules returned relevant answers in the tested documents. When the relevant content was absent, the assistant stated that the documents did not specify it. Additional synthetic documents were used to test disciplinary actions, fraud reporting, Admin-only access, and Finance department retrieval.

The results reflect local manual tests and are not a formal security audit or benchmark. Before final submission, update this section with any further test cases and outcomes that you personally verify.

7. Limitations

• Retrieval performance depends on extraction quality, chunk boundaries, embedding model, and ranking.
• Scanned PDFs may need OCR support.
• Conflicting policy documents can produce mixed context; policy versioning and explicit conflict handling are future improvements.
• Local persistence and restart behavior depend on the implementation and should be documented based on actual tests.
• The prototype’s authentication and access controls are not certified for production use.

8. Future Scope

Potential extensions include OCR, hybrid keyword/vector retrieval, reranking, systematic evaluation datasets, policy version tracking, stronger authentication, audit logging, additional document formats, and more robust conflict detection.

9. Conclusion

Nexus AI demonstrates a practical retrieval-augmented generation workflow for querying policy PDFs. It combines document processing, vector retrieval, metadata-based access filtering, LLM response generation, and source citations in a local Streamlit application. The project is suitable for an academic or portfolio demonstration, with production deployment and security hardening intentionally outside its current scope.

Appendix — Repository checklist

Before publishing to GitHub, inspect the code and repository for API keys, .env files, uploaded confidential PDFs, local vector indexes, user data, and absolute local paths. Include requirements.txt and a clear setup guide. Confirm all included sample policies are fictional and shareable. Add a license only if you choose to permit reuse.